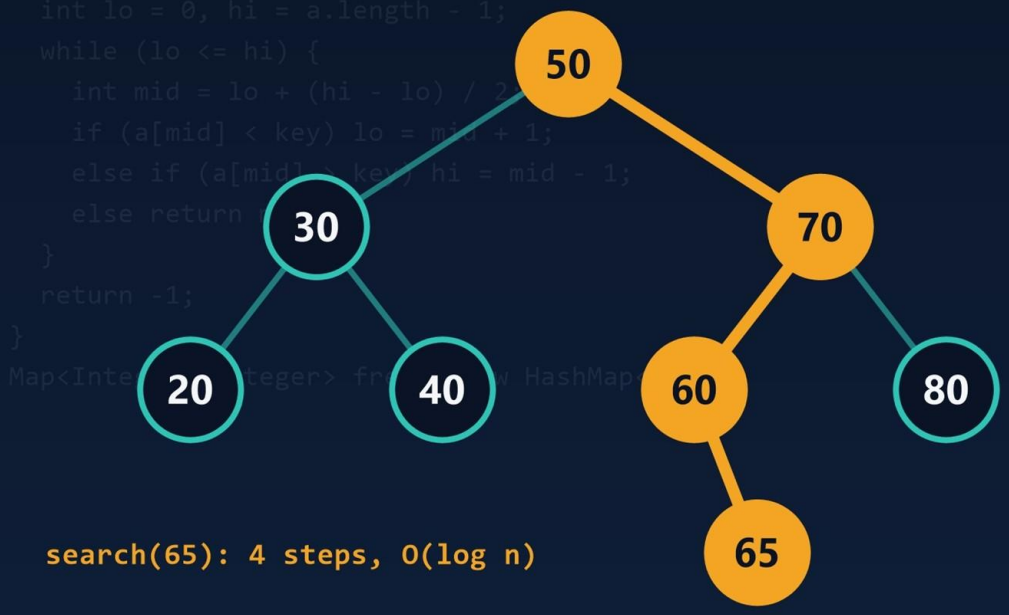
SECOND EDITION



THE CODE INTERVIEW

Real Java Coding Interview Questions, Solved Step by Step

48 real questions · brute force → optimal · 117 follow-ups

MOISES GAMIO
Founder of codersite.dev

FREE SAMPLE · CHAPTER 1

You have 45 minutes and a whiteboard. Be ready.

You build real software every day. But the coding interview tests something else: solving an unfamiliar problem under time pressure, out loud, with an interviewer watching every line you write. The Code Interview prepares you for exactly that moment.

Inside this free sample

- **The complete Chapter 1, Arrays and Strings:** 8 real interview questions, from reversing a text to Items in Closed Compartments, DHL Locker Locations and Largest Sibling of a Number.
- **The full table of contents**, so you can see everything the book covers.

What the full book gives you

- **48 real interview questions** in 10 chapters: arrays, linked lists, math puzzles, recursion, sorting and searching, stacks and queues, hash tables, trees, graphs and complete coding challenges.
- **One proven path for every question:** clarifying questions, brute force, an efficient solution with every variable explained, a step-by-step walkthrough, JUnit tests, Big O and an interview tip.
- **117 follow-up questions with answers:** overflow, huge inputs, duplicates, thread safety, and the JDK classes used in production.
- **Business-style problems** like the ones companies really ask: inventory compartments, lockers, shopping budgets and bakery orders.
- **Every listing tested again** for this Second Edition, plus study plans for one weekend, two weeks or four weeks.

→ **Get the full book on Amazon**

Kindle and paperback · As an Amazon Associate I earn from qualifying purchases.

THE CODE INTERVIEW

Real Java Coding Interview Questions, Solved Step by Step

Second Edition

Moises Gamio

Founder, codersite.dev

The Code Interview

by Moises Gamio

Second Edition

Copyright © 2026 Moises Gamio. All rights reserved.

No part of this book may be reproduced, stored in a retrieval system, or transmitted in any form or by any means—electronic, mechanical, photocopying, recording, or otherwise—without the prior written permission of the publisher.

Thank you for reading! If this book helps you prepare, please take one minute to leave a review on the store where you bought it. Reviews help other developers find the book and help me improve the next edition. Found an error in the text or the code, or have a suggestion? Email me at codersitedev@gmail.com. I read every message.

Cover designer: Moises Gamio

ISBN: 9798177831381

For Lorena

Contents

This free sample includes the complete Chapter 1, Arrays and Strings (pages 10-40). The full book contains every chapter listed below.

PREFACE vii

What's New in This Edition viii

Who This Book Is For x

How to Use This Book xi

Introduction 1

Data structures and Algorithms 1

Big O Notation 1

Arrays and Strings 10

1.1 Reverse a Text 12

1.2 Validate If a String Has All Unique Characters 15

1.3 Validate If a String Is a Palindrome 17

1.4 Compare Application Version Numbers 19

1.5 Rotate a Matrix by 90 Degrees 23

1.6 Items in Closed Compartments 25

1.7 DHL Locker Locations 31

1.8 Largest Sibling of a Number 37

Lists and Linked Lists 41

2.1 Remove Duplicates from a List 43

2.2 Implement a Linked List 45

2.3 Reverse a Linked List 48

2.4 Design and implement LRU cache 50

Math and Logic Puzzles 58

3.1 Sum of All Even Numbers from 1 to a Given Integer 59

3.2 Validate a Leap Year 60

3.3 Given an Integer N, Returns the Smallest Number of Digits 62

3.4 Fizz-Buzz 64

3.5 Verify If a Number Is a Valid Power of a Base 67

3.6 Validate If a Number Is Prime or Not 69

3.7 Distance Between Two Points 71

3.8 Write an Immutable Class to convert Currencies 73

3.9 Number of Products of Two Consecutive Integers 76

3.10 Assemble Parts in Minimum Time 78

Recursion 82

4.1 Calculate Factorial of a Given Integer N 82

4.2 Calculate Fibonacci Series 84

4.3 Generate All Permutations of Characters in a Given Text 86

4.4 Greatest Common Divisor 90

Sorting and Searching **97**
5.1 Bubble Sort *97*
5.2 Insertion Sort *100*
5.3 Quick Sort *102*
5.4 Binary Search *104*
5.5 Merge Two Sorted Lists *106*
5.6 Shopping Options *109*
5.7 Find the missing term in an Arithmetic Progression *113*
Stacks and Queues **119**
6.1 Delimiter Matching *120*
6.2 Queue via Stacks *122*
6.3 Reverse a String using a Stack *125*
Hash Table **127**
7.1 Design a Hash Table *128*
7.2 Find the Most Frequent Elements in an Array *132*
7.3 Nuts and Bolts *134*
Trees **136**
8.1 Binary Search Tree *137*
8.2 Tree Traversal *141*
Graphs **145**
9.1 Depth-First Search (DFS) *146*
9.2 Breadth-First Search (BFS) *152*
9.3 Social Network Traversal *157*
Coding Challenges **166**
10.1 Optimize Online Purchases *166*
10.2 Tic Tac Toe *176*
10.3 Maximum Waiting Time at a Filling Car Station *195*
10.4 Bakery - Cost and Packaging Breakdown for each Product *201*
Appendix **210**
Coding Best Practices *210*
Basics of Generics and Functional Programming *217*

CHAPTER 1

Arrays and Strings

1.0 Fundamentals

Arrays

An array is a container object that stores a fixed number of elements of the same data type under a single name. It uses a contiguous memory location to store the elements. Its numerical index accesses each element.

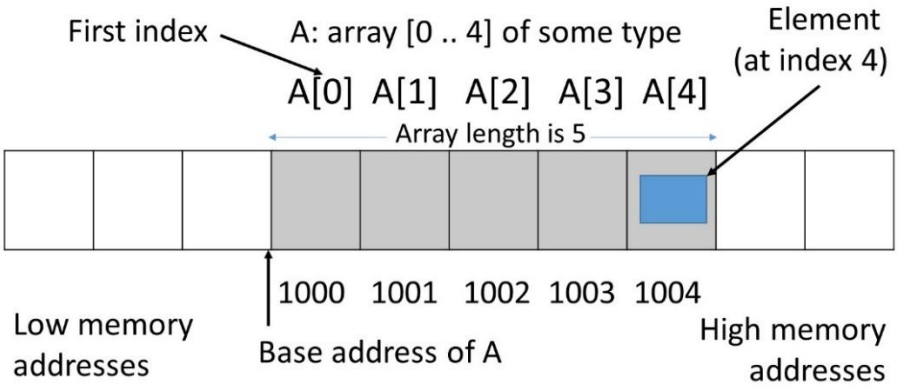


Figure 1.0.1 Array

If you ask the Array, give me the element at index 4, the computer locates that element's cell in a single step.

That happens because the computer finds the memory address where the Array begins - 1000 in the figure above - and adds 4, so the element will be located at memory address 1004.

Arrays are a *linear data structure* because the elements are arranged sequentially and accessed randomly.

Characteristics of Arrays:

- Homogeneous Data: All elements in an array must be of the same data type, such as integers, strings, or floating-point numbers.

To declare an array in Java, the following syntax is used:

```
int[] arrayOfInts; //Declaration
```

- Static Size: Arrays have a fixed size at the time of declaration. Once an array is created, its size cannot be changed without creating a new array.

```
//allocates memory for 10 integers
arrayOfInts = new int[10]; //Initialization

//initialize fourth element
arrayOfInts[3] = 200;
```

Alternatively, both declaration and initialization can be done in a single line:

```
int[] numbers = {1, 2, 3, 4, 5};
```

- Ordered Collection: Arrays store elements in a specific order, and each element has a unique index.

Common Operations on Arrays:

- Accessing Elements: Arrays can be accessed using their indices.

```
//get contents of fourth element
int temp = arrayOfInts[3]; //Output: 200
```

In a multidimensional array, you retrieve an array when accessing each index. A two-dimensional array is known as a matrix.

Arrays allow us to have one variable store multiple values.

For example, we define an array of products 2-D, whose elements per row represents: *productId, price,* and *grams.*

```
double[][] arrayOfProducts = new double[4][3];
arrayOfProducts[2][0] = 2;
arrayOfProducts[2][1] = 1.29;
arrayOfProducts[2][2] = 240;
```

The following figure shows the representation of this Array of arrays:

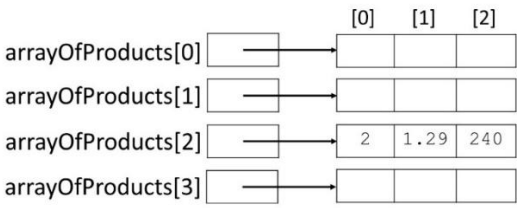


Figure 1.0.2 Multidimensional Array

- Insertion and Deletion: Insertion and deletion of elements are not efficient in arrays due to their static size. However, you can insert or delete elements by creating a new array and copying the relevant elements.

- Traversing Arrays: Traversing an array involves iterating through all its elements, typically using a for loop or a while loop.

```
for (int element: arrayOfInts) {
  System.out.println(element);
}
```

Array Methods

Java provides utility methods via the *Arrays* class in the *java.util* package.

```
import java.util.Arrays;
int[] nums = {4, 2, 9, 1};
Arrays.sort(nums); // Sorting the array
System.out.println(Arrays.toString(nums)); // Output: [1, 2, 4, 9]
```

Use Arrays when you need **fast access** to elements and a **fixed-size** structure

Strings

A string in Java is an object that represents a sequence of characters. Strings are immutable, meaning once created, their values cannot be changed.

String Declaration and Initialization

Strings can be created in multiple ways:

```
String s1 = "Hello"; // String literal
String s2 = new String("Hello"); // Using the new keyword
```

String Comparison

To compare strings, the equals() method should be used instead of ==

```
String s1 = "Java";
String s2 = "Java";
System.out.println(s1.equals(s2)); // Output: true
System.out.println(s1 == s2); // Output: true (for literals, due to string pooling)
```

StringBuffer and StringBuilder

For mutable strings, Java provides StringBuffer and StringBuilder. The main difference is that StringBuffer is synchronized (thread-safe) while StringBuilder is not.

```
StringBuilder sb = new StringBuilder("Hello");
sb.append(" World");
System.out.println(sb.toString()); // Output: Hello World
```

Arrays and strings are fundamental data structures in Java that provide powerful methods for data manipulation and storage. While arrays are efficient for fixed-size collections, strings offer a flexible way to handle text data. Understanding how to utilize both effectively is essential for any Java interview.

1.1 Reverse a Text

Given a string of characters, reverse the order of the characters in an efficient manner.

Clarifying Questions

- **Can the input be null or empty?** Our method returns null for a null input and an empty string for an empty one.
- **May I use built-in methods such as StringBuilder.reverse()?** Mention it, because it shows you know the API, but interviewers usually want you to implement the logic yourself.
- **Does the text contain only simple characters?** We assume that each character fits in a single char. Emojis and some other Unicode symbols use two chars (a surrogate pair), and a naive swap would break them.

Brute-Force Solution

The most obvious approach builds a new string by adding each character in front of the result:

```
String result = "";
for (char c : text.toCharArray()) {
  result = c + result; // creates a new String every time
}
return result;
```

It works, but every concatenation copies the whole string built so far, because Java strings are immutable. For a text of n characters, that is 1 + 2 + … + n character copies, which is $O(n^2)$ time. The efficient solution avoids these copies.

Efficient Solution

We choose an array of characters as our data structure. A Java String is immutable, so we cannot change it directly, but a char array lets us read and swap any position in constant time using its index.

Firstly, convert the text to be reversed to a character array. Then, calculate the length of the string.

Secondly, swap the elements in place with a loop: the first character with the last one, the second with the second-to-last, and so on, moving inward. Each swap uses a single temporary variable, so the reversal itself needs no extra memory, and we only need to iterate through the first half of the array.

Finally, it returns the new character array as a String. Listing 1.1 shows the algorithm.

Key variables in Listing 1.1:

- **chars:** a mutable copy of the text, which we reverse in place.
- **n:** the number of characters.
- **idx:** the position on the left side. Its mirror position on the right side is n - 1 - idx.
- **temp:** holds one character during a swap, so that it is not lost.

Listing 1.1 – Reverse a Text

```
public class StringUtils {
  public static String reverse(String text) {
    if (text == null)
       return null;
    char[] chars =text.toCharArray();
    final int n =chars.length;
    char temp;
    for (int idx =0; idx < n/2; idx++) {
      temp =chars[idx];
      chars[idx] =chars[n - 1 - idx];
      chars[n - 1 - idx] =temp;
    }
    return String.valueOf(chars);
  }
}
```

Walkthrough:

Let's trace reverse("abc2132"). Here n = 7, so the loop runs while idx < 3, and the middle character (index 3) stays in place:

```
When idx = 0:
chars = {a, b, c, 2, 1, 3, 2}
chars[idx] = a
chars[n-1-idx] = 2

When idx = 1:
chars = {2, b, c, 2, 1, 3, a}
chars[idx] = b
chars[n-1-idx] = 3

When idx = 2:
chars = {2, 3, c, 2, 1, b, a}
chars[idx] = c
chars[n-1-idx] = 1

When idx = 3:
chars = {2, 3, 1, 2, c, b, a}
idx is not less than n/2
end
```

Tests

```
@Test
public void reverseText_useCases() {
  assertEquals("abc2132", StringUtils.reverse("2312cba"));
  assertEquals("ba", StringUtils.reverse("ab"));
  assertEquals("c a1", StringUtils.reverse("1a c"));
}
```

Complexity Analysis

Time complexity: O(n). We perform n/2 swaps, and each character is visited once, where n is the length of the string.

Space complexity: O(n). toCharArray() creates a copy of the string, because Java strings

are immutable. The swapping itself uses only O(1) extra memory.

Follow-Up Questions

- **How do you reverse the order of the words in a sentence, for example "Java is fun" into "fun is Java"?** Reverse the whole string, then reverse each word in place. Alternatively, split the sentence on spaces and join the words in reverse order.
- **Can you solve it recursively?** Yes: reverse(s) = reverse(s.substring(1)) + s.charAt(0). It is short, but it takes $O(n^2)$ time because of the string copies and O(n) stack space, which is a good argument for the iterative version.
- **How do you handle surrogate pairs?** Use new StringBuilder(text).reverse(), which keeps each surrogate pair in the correct order.

Interview Tip: swapping from both ends toward the middle is the key idea. It reappears in palindromes (question 1.3), array rotations, and many two-pointer problems.

1.2 Validate If a String Has All Unique Characters

Given a string of characters, validate if all of them are unique.

Clarifying Questions

- **Which character set does the string use?** Always ask. We assume ASCII (128 characters), which determines the size of our lookup array.
- **Is the comparison case-sensitive?** We assume it is: 'a' and 'A' are different characters.
- **Can the string be null?** We assume a non-null input.

Brute-Force Solution

Compare every character with every character after it:

```
for (int i = 0; i < str.length(); i++) {
  for (int j = i + 1; j < str.length(); j++) {
    if (str.charAt(i) == str.charAt(j))
      return false;
  }
}
return true;
```

No extra memory is needed, but the two nested loops make about $n^2/2$ comparisons, which is $O(n^2)$ time. We can do better by remembering which characters we have already seen.

Efficient Solution

We assume the charset is ASCII, but you should always ask interviewers if you are unsure. Originally based on the English alphabet, ASCII encodes 128 specified characters into

seven-bit integers. Ninety-five of the encoded characters are printable: these include the digits *0* to *9*, lowercase letters *a* to *z*, uppercase letters *A* to *Z*, and punctuation symbols.

Firstly, create a Boolean array to store the occurrence of every character.

Secondly, iterate over the string. For each character, use its numeric code as an index into the Boolean array. If that position is already true, we have seen the character before, so the string does not have all unique characters. Otherwise, mark the position as true.

Finally, if all characters had only one occurrence in the Boolean Array, the string has all unique characters. Listing 1.2 shows the algorithm.

Key variables in Listing 1.2:

- **booleans:** an array of 128 flags, one for each ASCII code. For example, booleans[65] is true once 'A' has been seen.
- **value:** the numeric code of the current character. Assigning a char to an int gives its code: '2' is 50 and 'a' is 97.
- **The length check:** a string with more than 128 characters must repeat at least one of the 128 ASCII characters (the pigeonhole principle), so we can return false immediately.

Walkthrough: for areUniqueChars("29s2"):

```
'2' -> value 50  -> booleans[50] is false -> mark it true
'9' -> value 57  -> booleans[57] is false -> mark it true
's' -> value 115 -> booleans[115] is false -> mark it true
'2' -> value 50  -> booleans[50] is true  -> return false
```

Listing 1.2 – Validate if a String Has All Unique Characters

```
public class StringUtils {
  public static boolean areUniqueChars(String str) {
    if (str.length() > 128)
      return false;

    boolean[] booleans = new boolean[128];
    for (int idx = 0; idx < str.length(); idx++) {
      int value = str.charAt(idx);
      if (booleans[value]) { //is found?
        return false;
      }
      booleans[value] = true;
    }
    return true;
  }
}
```

Tests

```
public class AreUniqueCharsTest {
  @Test
  public void is_not_UniqueChars() {
    assertFalse(StringUtils.areUniqueChars("29s2"));
    assertFalse(StringUtils.areUniqueChars("1903aio9p"));
```

```
  }
  @Test
  public void is_UniqueChars() {
    assertTrue(StringUtils.areUniqueChars("29s13"));
    assertTrue(StringUtils.areUniqueChars("2813450769"));
  }
}
```

Complexity Analysis

Time complexity: O(n). We visit each character once. Because any string longer than 128 characters is rejected immediately, the loop never runs more than 128 times, so for a fixed character set you can also call it O(1).

Space complexity: O(1). The boolean array always has 128 elements, regardless of the input size.

Follow-Up Questions

- **What if you cannot use an additional data structure?** Compare every character with every other one, in $O(n^2)$ time and O(1) space, or sort the characters and compare neighbors, in O(n log n) time.
- **What if the string is Unicode instead of ASCII?** A fixed array of 128 elements is no longer enough. Use a HashSet of characters (or code points): O(n) time and O(n) space.
- **What if the string contains only the lowercase letters a to z?** A single int can work as a bit vector of 26 bits, reducing the extra memory to one integer.

> **Interview Tip:** when the set of possible values is small and known, an array indexed by the value is faster and simpler than a hash table.

1.3 Validate If a String Is a Palindrome

A palindrome is a string that reads the same forward and backward, for example, *level, wow,* and *madam.*

Clarifying Questions

- **Is the check case-sensitive?** We assume it is: “Madam” is not a palindrome, but “madam” is.
- **Should spaces and punctuation be ignored?** Not in this version. See the follow-up questions.
- **What about null or an empty string?** A null input returns false. An empty string or a single character is a palindrome.

Brute-Force Solution

Reuse question 1.1: reverse the string and compare it with the original.

```
return text.equals(StringUtils.reverse(text));
```

This takes O(n) time, but it creates a reversed copy (O(n) extra space) and always processes the whole string, even when the first and last characters are already different. The efficient solution compares the characters in place and stops at the first mismatch.

Efficient Solution

Compare the first character with the last one, the second with the second-to-last, and so on, moving toward the middle. If any pair is different, the text is not a palindrome, and we can stop immediately. Listing 1.3 shows the algorithm.

Key variables in Listing 1.3:

- **length:** the number of characters in the text.
- **idx:** the position on the left side. Its mirror position on the right side is length - 1 - idx.
- **length / 2:** the number of pairs to compare. When the length is odd, the middle character has no partner and is not compared, because it always matches itself.

Walkthrough: for isPalindrome("2f1f2"), length = 5, so we compare two pairs:

```
idx = 0: charAt(0) = '2', charAt(4) = '2' -> equal
idx = 1: charAt(1) = 'f', charAt(3) = 'f' -> equal
idx = 2 is not less than 5 / 2 -> loop ends -> return true
```

For isPalindrome("2f1"), the first pair is already different ('2' and '1'), so the method returns false after a single comparison.

Listing 1.3 – Validate If a String Is a Palindrome

```
public class StringUtils {
  public static boolean isPalindrome(String text) {
    if (text == null)
      return false;

    final int length = text.length();
    for (int idx = 0; idx < length / 2; idx++) {
      if (text.charAt(idx) != text.charAt(length - 1 - idx))
        return false;
    }
    return true;
  }
}
```

Tests

```
public class IsPalindromeTest {
  @Test
  public void is_not_palindrome() {
    assertFalse(StringUtils.isPalindrome("2f1"));
    assertFalse(StringUtils.isPalindrome("-101"));
  }

  @Test
  public void is_palindrome() {
```

```
    assertTrue(StringUtils.isPalindrome("2f1f2"));
    assertTrue(StringUtils.isPalindrome("-101-"));
    assertTrue(StringUtils.isPalindrome("9"));
    assertTrue(StringUtils.isPalindrome("99"));
    assertTrue(StringUtils.isPalindrome("madam"));
  }
}
```

Complexity Analysis

Time complexity: O(n). We compare at most n/2 pairs of characters.

Space complexity: O(1). We only use an index variable.

Follow-Up Questions

- **How would you ignore case, spaces, and punctuation, as in "A man, a plan, a canal: Panama"?** Use two pointers, one at each end. Skip characters that are not letters or digits (Character.isLetterOrDigit) and compare the rest with Character.toLowerCase. It is still O(n) time and O(1) space.
- **Can you solve it recursively?** Yes: compare the first and last characters and recurse on the substring between them. It is elegant, but it uses O(n) stack space.
- **Can any permutation of the string form a palindrome?** Count the frequency of each character. At most one character may appear an odd number of times.

> **Interview Tip:** two pointers moving toward each other are a classic pattern for strings and sorted arrays. When you see "compare from both ends", think of two pointers.

1.4 Compare Application Version Numbers

Semantic versioning is a formal convention for specifying compatibility using a three-part version number: major, minor, and patch. The major version changes when the application programming interface (API) breaks backward compatibility, the minor version changes when new backward-compatible features are added, and the patch version changes for backward-compatible bug fixes. Given two version strings, version1 and version2, return:

* -1 if version1 < version2

* 1 if version1 > version2

* 0 if version1 == version2

Clarifying Questions

- **Can the versions have a different number of parts, such as "10.1" and "10.1.0"?** Yes. Missing parts count as zero, so these two versions are equal.
- **Can a part have leading zeros, such as "1.01"?** Yes. Integer.parseInt treats "01" as 1.

- **Can a version include a label, such as "1.0.0-beta"?** Not in this version. See the follow-up questions.

A Tempting Mistake

Why not simply compare the two strings with version1.compareTo(version2)? Because strings are compared character by character, not number by number:

```
"10.1".compareTo("9.1") // negative: '1' < '9', so 10.1 looks smaller than 9.1!
```

Version numbers must be compared as integers, part by part. That is exactly what the efficient solution does.

Efficient Solution

We need to compare two semantic version numbers of the form:

```
major.minor.patch
```

Example versions:

- "10.1.2"

- "10.1.1"

The comparison rules are:

1. Compare **major** numbers.
2. If equal, compare **minor** numbers.
3. If still equal, compare **patch** numbers.
4. If all are equal → return 0.

Approach

1. Split both version strings by ".".

   Example: "10.1.2" → [10, 1, 2]

2. Iterate through both arrays simultaneously.
   - Convert each part to an integer (Integer.parseInt).
   - Compare corresponding values:
     i. If v1 < v2 → return -1.
     ii. If v1 > v2 → return 1.
3. If no differences are found → return 0.

Why split("\\."): The split() method takes a regular expression, and in a regular expression a dot means "any character". The two backslashes escape the dot, so we split only on the literal '.' character.

Key variables in Listing 1.4:

- **v1Parts, v2Parts:** the parts of each version as strings, for example ["10", "1", "2"].
- **length:** the number of parts in the longer version, so that no part is skipped.
- **num1, num2:** the numeric values of the current parts. A missing part counts as 0, which makes "10.1" equal to "10.1.0".

Walkthrough: for compareVersionNumbers("10.1", "10.1.1"):

```
v1Parts = ["10", "1"], v2Parts = ["10", "1", "1"], length = 3
i = 0: num1 = 10, num2 = 10 -> equal, continue
i = 1: num1 = 1,  num2 = 1  -> equal, continue
i = 2: num1 = 0 (missing), num2 = 1 -> num1 < num2 -> return -1
```

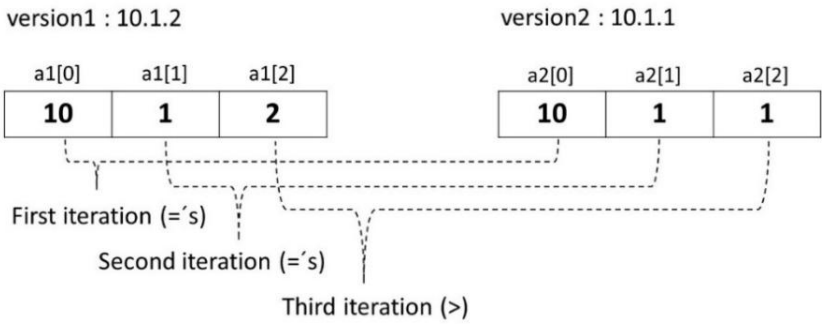


Figure 1.4 Compare Application Version Numbers

Listing 1.4 – Compare Application Version Numbers

```
public class VersionComparator {

  public static int compareVersionNumbers(String version1, String version2) {
    // Split version strings into parts
    String[] v1Parts = version1.split("\\.");
    String[] v2Parts = version2.split("\\.");

    // Determine the maximum length
    int length = Math.max(v1Parts.length, v2Parts.length);

    // Compare each part
    for (int i = 0; i < length; i++) {
      // Missing parts are considered as 0
      int num1 = (i < v1Parts.length) ? Integer.parseInt(v1Parts[i]) : 0;
      int num2 = (i < v2Parts.length) ? Integer.parseInt(v2Parts[i]) : 0;

      if (num1 < num2) {
        return -1;
      } else if (num1 > num2) {
        return 1;
      }
```

```
    }

    // Versions are equal
    return 0;
  }
}
```

Tests

```
@Test
public void versionNumber_usesCases() {
  assertEquals(0, VersionComparator.compareVersionNumbers("15", "15.0"));
  assertEquals(0, VersionComparator.compareVersionNumbers ("10.1", "10.1.0"));
  assertEquals(-1, VersionComparator.compareVersionNumbers ("10.1", "10.1.1"));
  assertEquals(1, VersionComparator.compareVersionNumbers ("10.1.2", "10.1.1"));
}
```

Complexity Analysis

Time Complexity:

- Splitting each string costs O(n + m), where n and m are the lengths of the two strings.
- Comparing each version part takes O(k), where k is the number of version components (usually 3 for semantic versioning, but could be more).
- Total: O(n + m + k) → linear in the input size.

Space Complexity:

- We store two arrays of version parts → O(k).
- No extra large structures are used.

For the standard case of semantic versioning (major.minor.patch), k = 3, so the complexity is practically O(n + m), dominated by splitting the strings.

Follow-Up Questions

- **How would you support pre-release labels, such as "2.0.0-alpha"?** Split off the label after the "-". Compare the numbers first; if they are equal, a version without a label is greater than one with a label (2.0.0 > 2.0.0-alpha).
- **What if a part is larger than Integer.MAX_VALUE?** Integer.parseInt throws an exception. Compare the parts as strings instead: remove leading zeros, and the longer string is the larger number; if both have the same length, compare them alphabetically.
- **Can you avoid split() and its extra arrays?** Yes. Walk through both strings with two indexes, building each number digit by digit. The extra space becomes O(1).

> **Interview Tip:** real-world questions like this one test your attention to detail. List the edge cases ("15" versus "15.0", leading zeros) before coding; the interviewer is often waiting for exactly that.

1.5 Rotate a Matrix by 90 Degrees

Given a square matrix, turn it by 90 degrees in a clockwise direction.

Clarifying Questions

- **Is the matrix always square (n × n)?** Yes. Only a square matrix can be rotated in place.
- **Must the rotation be done in place?** Yes, with O(1) extra memory. Otherwise, we could simply copy each element into a new matrix: result[j][n - 1 - i] = matrix[i][j].
- **Clockwise or counterclockwise?** Clockwise.

Brute-Force Solution

If extra memory is allowed, copy each element to its rotated position in a new matrix. The element in row i and column j moves to row j and column n - 1 - i:

```
int[][] rotated = new int[n][n];
for (int i = 0; i < n; i++)
  for (int j = 0; j < n; j++)
    rotated[j][n - 1 - i] = matrix[i][j];
```

This is simple and takes $O(n^2)$ time, but it needs $O(n^2)$ extra memory for the second matrix. Interviewers usually ask for an in-place rotation.

Efficient Solution

We rotate the matrix layer by layer, like the rings of an onion. The outer loop visits each layer, from the outside in; an n × n matrix has n / 2 layers (a 3 × 3 matrix has one layer, and its center cell never moves). The inner loop walks along the top edge of the layer. Each step moves four cells at once, one from each side, using a single temporary variable. Listing 1.5 shows the algorithm.

Key variables in Listing 1.5:

- **i:** the current layer. Layer 0 is the outer ring.
- **j:** the position along the edge of the layer, from i to n - i - 2.
- **The four cells moved in one step:** top = matrix[i][j], right = matrix[j][n - 1 - i], bottom = matrix[n - 1 - i][n - 1 - j], and left = matrix[n - 1 - j][i]. The code comments call them top-left, top-right, bottom-right, and bottom-left.
- **temp:** saves the top cell, because it is overwritten first.

In a clockwise rotation, left moves to top, bottom moves to left, right moves to bottom, and top (saved in temp) moves to right.

Walkthrough: a 3 × 3 matrix has one layer (i = 0), and j takes the values 0 and 1:

```
start        after j = 0   after j = 1
[1, 2, 3]    [7, 2, 1]     [7, 4, 1]
```

```
[4, 5, 6]    [4, 5, 6]     [8, 5, 2]
[7, 8, 9]    [9, 8, 3]     [9, 6, 3]
```

With j = 0, the four corners rotate (1, 3, 9, 7). With j = 1, the four edge centers rotate (2, 6, 8, 4). The result is the original matrix turned 90 degrees clockwise.

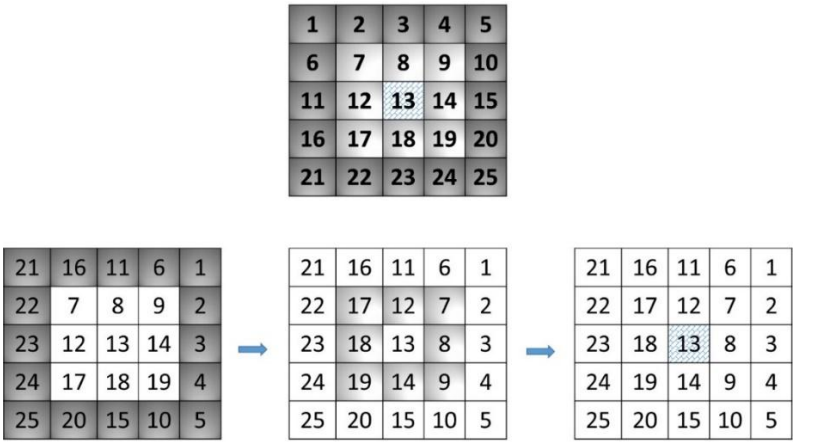


Figure 1.5 Rotate a Matrix by 90 Degrees

Listing 1.5 – Rotate a Matrix by 90 Degrees.

```
public class MatrixUtils {
  public static void rotate(int[][] matrix) {
    int n = matrix.length;
    if (n <=1)
      return;
    /* layers */
    for (int i = 0; i < n / 2; i++) {
      /* elements */
      for (int j = i; j < n - i - 1; j++) {
        //Swap elements in the clockwise direction
        //temp = top-left
        int temp = matrix[i][j];
        //top-left <- bottom-left
        matrix[i][j] = matrix[n - 1 - j][i];
        //bottom-left <- bottom-right
        matrix[n - 1 - j][i] = matrix[n - 1 - i][n - 1 - j];
        //bottom-right <- top-right
        matrix[n - 1 - i][n - 1 - j] = matrix[j][n - 1 - i];
        //top-right <- top-left
        matrix[j][n - 1 - i] = temp;
      }
    }
  }
}
```

Tests

```
public class MatrixUtilsTest {

  @Test
  public void rotate4x4() {
    int[][] matrix = new int[][]{
```

```
        {9, 10, 11, 12},
        {16, 17, 18, 19},
        {23, 24, 25, 26},
        {30, 31, 32, 33}};
    MatrixUtils.rotate(matrix);
    assertArrayEquals(new int[]{30, 23, 16, 9}, matrix[0]);
    assertArrayEquals(new int[]{33, 26, 19, 12}, matrix[3]);
  }

  @Test
  public void rotate5x5() {
    int[][] matrix = new int[][]{
      {1, 2, 3, 4, 5},
      {6, 7, 8, 9, 10},
      {11, 12, 13, 14, 15},
      {16, 17, 18, 19, 20},
      {21, 22, 23, 24, 25}};
    MatrixUtils.rotate(matrix);
    assertArrayEquals(new int[]{21, 16, 11, 6, 1}, matrix[0]);
    assertArrayEquals(new int[]{22, 17, 12, 7, 2}, matrix[1]);
  }
}
```

Complexity Analysis

Time complexity: $O(n^2)$. Every element of the n × n matrix is moved exactly once.

Space complexity: O(1). We only use one temporary variable.

Follow-Up Questions

- **Is there a simpler way to rotate the matrix?** Yes: transpose the matrix (swap matrix[i][j] with matrix[j][i]), then reverse each row. You get two simple loops with the same $O(n^2)$ time and O(1) space.
- **How do you rotate counterclockwise?** Transpose the matrix, then reverse each column instead of each row.
- **How do you rotate by 180 degrees?** Reverse the order of the rows, then reverse each row.

Interview Tip: draw a 4 × 4 matrix and follow a single element around the four corners before you write any index expression. Index mistakes are the most common bug in matrix questions.

1.6 Items in Closed Compartments

ACME Corporation would like to know how much inventory exists in their closed inventory compartments. Given a string *s* consisting of items as "*" and closed compartments as an open and close "|", an array of starting indices *startIndices* and an array of ending indices *endIndices*, determine the number of items in closed compartments within the substring between the two indices, inclusive.

- An item is represented as an asterisk ('*' = ascii decimal 42)

- A compartment is represented as a pair of pipes that may or may not have items between them ('|' = ascii decimal 124).

Example

s = '|**|*|*'

startIndices = [1,1]

endIndices = [5,6]

The string has a total of 2 closed compartments, one with 2 items and one with 1 item. For the first pair of indices, *(1,5),* the substring is '|**|*'. There are 2 items in a compartment.

For the second pair of indices, *(1,6),* the substring is '|**|*|' and there are *2 + 1 = 3* items in compartments.

Both of the answers are returned in an array. *[2, 3].*

Function Description

Write a function that returns an integer array that contains the results for each of the *startIndices[i]* and *endIndices[i]* pairs.

The function must have three parameters:

- *s*: A string to evaluate
- *startIndices*: An integer array, the starting indices.
- *endIndices*: An integer array, the ending indices.

Constraints (n is the length of s, and m is the number of queries):

- $1 \leq m, n \leq 10^5$
- $1 \leq startIndices[i] \leq endIndices[i] \leq n$
- Each character of *s* is either '*' or '|'

Clarifying Questions

- **Are the indices 0-based or 1-based?** They are 1-based, as the example shows, so we subtract 1 before using them.
- **Must both walls of a compartment be inside the substring?** Yes. Items are counted only between two '|' characters that are both inside the range [start, end].
- **How many queries can there be?** Up to 100,000, which rules out scanning the substring again for every query.

Understanding the Problem

Problem Summary

Given:

- A string s of '*' (items) and '|' (compartment walls),

- Two arrays *startIndices* and *endIndices* representing query ranges (1-based indices).

Goal:

- For each (start, end) pair, return the number of '*' characters that are inside closed compartments — i.e., between matched '|' characters.

Key Observations

- Items outside of compartment boundaries (not between '|'s) do not count.

- Each query should look only inside the specified substring, but doing this for every query from scratch could be too slow (O(n × m)).

Brute-Force Solution

In a real interview, most candidates first try the simplest solution: for each query, build the substring and scan it from left to right. We need two counters and a flag:

- **pending:** the items seen since the last wall. They are not counted yet, because we do not know whether another wall will close their compartment.
- **total:** the items in compartments that are already closed. When we find a wall, pending items move into total.
- **wallFound:** becomes true at the first wall. Items before the first wall can never be inside a compartment.

```
public static List<Integer> bruteForce(String s,
    List<Integer> startIndices, List<Integer> endIndices) {
  List<Integer> result = new ArrayList<>();
  for (int q = 0; q < startIndices.size(); q++) {
    String sub = s.substring(startIndices.get(q) - 1, endIndices.get(q));
    int total = 0, pending = 0;
    boolean wallFound = false;
    for (char c : sub.toCharArray()) {
      if (c == '|') {
        if (wallFound) total += pending; // a compartment closes
        wallFound = true;
        pending = 0;
      } else {
        pending++;
      }
    }
    result.add(total);
  }
  return result;
}
```

It passes the examples, but every query scans up to n characters again. With n = 100,000 characters and m = 100,000 queries, that is up to 10 billion steps, and it times out on large hidden tests. The fix is to do the scanning work only once:

```
Approach      Preprocessing   Per query   Total
Brute force   none            O(n)        O(n × m)
Prefix sums   O(n)            O(1)        O(n + m)
```

Efficient Solution: Prefix Sums

We will precompute useful data to answer each query in O(1):

Key idea: if we know, for every position, how many items lie between the first wall and that position, then the number of items between any two walls is a simple subtraction. For each query, we only need to find the right two walls: the first wall inside the range and the last wall inside the range. Everything between them is inside closed compartments.

Step 1: asterisksSum[]

asterisksSum[i] stores the number of '*' characters from the first '|' of the string up to index i. Items before the first wall are never counted, because they can never be inside a compartment.

Step 2: leftPipe[] and rightPipe[]

leftPipe[i]: the index of the nearest '|' at or to the left of i, or -1 if there is none.

rightPipe[i]: the index of the nearest '|' at or to the right of i, or -1 if there is none.

Query Logic

For each query (start, end) (convert to 0-based indices):

1. Find the first '|' at or after start → left = rightPipe[start]
2. Find the **last '|'** at or before `end` → `right = leftPipe[end]`
3. If `left < right`, then the number of items between them is:

```
asterisksSum[right] - asterisksSum[left]
```

4. Otherwise, answer is 0.

Listing 1.6 – Items in Closed Compartments.

```
public class ACMEInventory {
  public static List<Integer> numberOfItemsInClosedCompartments(
    String s, List<Integer> startIndices, List<Integer> endIndices) {

    if (startIndices.size()<1 || startIndices.size()>100000)
      throw new RuntimeException("wrong size in startIndices");

    if (endIndices.size()<1 || endIndices.size()>100000)
      throw new RuntimeException("wrong size in endIndices");

    int n = s.length();
    int[] asterisksSum = new int[n];
    int[] leftPipe = new int[n];
```

```
    int[] rightPipe = new int[n];

    // Fill prefix sum of items inside compartments
    int itemCount = 0;
    boolean insideCompartment = false;

    for (int i = 0; i < n; i++) {
      char c = s.charAt(i);
      if (c == '|') {
        insideCompartment = true;
        asterisksSum[i] = itemCount;
      } else if (c == '*') {
        if (insideCompartment) {
          itemCount++;
        }
        asterisksSum[i] = itemCount;
      } else {
        asterisksSum[i] = itemCount;
      }
    }

    // Fill leftPipe[i]: nearest '|' to the left (or at i)
    int nearestLeftPipe = -1;
    for (int i = 0; i < n; i++) {
      if (s.charAt(i) == '|') {
        nearestLeftPipe = i;
      }
      leftPipe[i] = nearestLeftPipe;
    }

    // Fill rightPipe[i]: nearest '|' to the right (or at i)
    int nearestRightPipe = -1;
    for (int i = n -1; i >= 0; i--) {
      if (s.charAt(i) == '|') {
        nearestRightPipe = i;
      }
      rightPipe[i] = nearestRightPipe;
    }

    List<Integer> result = new ArrayList<>();

    for (int i = 0; i < startIndices.size(); i++) {
      int start = startIndices.get(i) - 1; // Convert to 0-based
      int end = endIndices.get(i) -1;

      int left = rightPipe[start];
      int right = leftPipe[end];

      if (left != -1 && right != -1 && left < right) {
        result.add(asterisksSum[right] - asterisksSum[left]);
      } else {
        result.add(0);
      }
    }

    return result;
  }
}
```

Figure 1.6 shows the precomputed arrays for the example string.

Precomputed arrays for s = "||*|*"**

index i	0	1	2	3	4	5	6
s	|	*	*	|	*	|	*
asterisksSum	0	1	2	2	3	3	4
leftPipe	0	0	0	3	3	5	5
rightPipe	0	3	3	3	5	5	-1

Shaded columns are walls (|). A query only reads asterisksSum at walls: (1,6) -> 3 - 0 = 3

Figure 1.6 Precomputed arrays for Items in Closed Compartments

Walkthrough: for s = '|**|*|*', the three arrays computed by Listing 1.6 are:

```
index i        0   1   2   3   4   5   6
s              |   *   *   |   *   |   *
asterisksSum   0   1   2   2   3   3   4
leftPipe       0   0   0   3   3   5   5
rightPipe      0   3   3   3   5   5  -1
```

The item at index 6 is counted in asterisksSum, even though no wall ever closes its compartment. That is not a problem: a query only reads asterisksSum at wall positions (0, 3, and 5), and at a wall, every counted item lies inside a closed compartment. Now let's answer some queries (the problem uses 1-based indexes, so we subtract 1 first):

- **Query (1, 5):** start = 0 and end = 4. The first wall at or after start is left = rightPipe[0] = 0, and the last wall at or before end is right = leftPipe[4] = 3. Since left < right, the answer is asterisksSum[3] - asterisksSum[0] = 2 - 0 = 2.
- **Query (1, 6):** start = 0 and end = 5, so left = rightPipe[0] = 0 and right = leftPipe[5] = 5. The answer is asterisksSum[5] - asterisksSum[0] = 3 - 0 = 3.
- **Query (2, 7):** start = 1 and end = 6. Now left = rightPipe[1] = 3, because the wall at index 0 is outside the range, and right = leftPipe[6] = 5. The answer is asterisksSum[5] - asterisksSum[3] = 3 - 2 = 1. The two items at indexes 1 and 2 are excluded, because their left wall is not in the substring.
- **Query (5, 7):** start = 4 and end = 6, so left = rightPipe[4] = 5 and right = leftPipe[6] = 5. Since left is not less than right, the range contains no complete compartment, and the answer is 0.

Tests

```
@Test
public void test_numberOfItems() {
```

```
  assertEquals(new ArrayList<Integer>(Arrays.asList(2,3)),
    ACMEInventory.numberOfItemsInClosedCompartments("|**|*|*|*",
      new ArrayList<Integer>(Arrays.asList(1,1)),
      new ArrayList<Integer>(Arrays.asList(5,6))
  ));
}
@Test
public void test_rangesThatCutCompartments() {
  assertEquals(Arrays.asList(1, 0),
    ACMEInventory.numberOfItemsInClosedCompartments("|**|*|*|*",
      Arrays.asList(2, 5),
      Arrays.asList(7, 7)));
}
```

Complexity Analysis

Time complexity: O(n + m), where n is the length of the string and m is the number of queries. Three linear passes build the arrays, and each query is answered in O(1).

Space complexity: O(n) for the three precomputed arrays, plus O(m) for the result list.

Follow-Up Questions

- **What is the brute-force solution?** For each query, scan the substring and count the items between walls: O(n × m) time, which is too slow for 100,000 queries on a long string.
- **What if the string can change between queries?** Prefix sums would have to be rebuilt after every update. Use a Fenwick tree (binary indexed tree) or a segment tree instead, which support updates and queries in O(log n).

> **Interview Tip:** many queries over the same data is a strong hint to precompute. Prefix sums turn each range query from O(n) into O(1).

1.7 DHL Locker Locations

DHL is currently underway putting DHL Lockers across Germany. The Locker team wants to ensure that lockers are placed so that customers in the city are always within a short distance from a DHL Locker. To account for this, they need a way to model locker placements and distances from lockers.

For this task you are provided:

1. A positive whole number range 1-9 representing the length of your city in city blocks

2. A positive whole number range 1-9 representing the width of your city in city blocks

3. An array containing all X coordinates representing DHL Locker locations, each X coordinate range 1-9

4. An array containing all Y coordinates representing DHL Locker locations, each Y coordinate range 1-9

Your job is to construct a 2-d grid of the city. Each element of the grid should be a positive whole number that specifies the number of blocks to the closest locker. The distance between two blocks is the sum of their horizontal and vertical distance (a move in the diagonal direction is therefore considered a distance of 2).

Example #1:

Given

```
1. 3
2. 5
3. 1
4. 1
```

Return

```
012
123
234
345
456
```

Example #2:

Given

```
1. 5
2. 7
3. 24
4. 37
```

Return

```
32345
21234
10123
21234
32323
43212
32101
```

Complete the function below.

```
static int[][] getLockerDistanceGrid(int cityLength, int cityWidth, int[] lock-
erXCoordinates, int[] lockerYCoordinates) {}
```

Clarifying Questions

- **Are the coordinates 1-based?** Yes. Locker (1, 1) is the top-left block, so we convert the coordinates to 0-based array indexes.
- **Which dimension is which?** The length is the number of columns (X), and the width is the number of rows (Y), as the example output shows.
- **Can the city have no lockers?** We assume there is at least one locker; otherwise, the distance would be undefined.

Understanding the Problem

Let's restate the problem precisely first, because most mistakes in this question come from mixing up rows, columns, and 1-based coordinates.

Problem Restatement

You are given:

- cityLength → number of columns (X dimension)
- cityWidth → number of rows (Y dimension)
- lockerXCoordinates[] → X positions of lockers
- lockerYCoordinates[] → Y positions of lockers

You must produce a **2D integer grid** where:

- Each cell (x, y) stores the **Manhattan distance** to the **nearest locker**.

Manhattan distance is defined as:

$$distance = |x_1 - x_2| + |y_1 - y_2|$$

Diagonal movement counts as 2 (because it is 1 horizontal + 1 vertical).

Coordinate System Clarification

The problem uses **1-based coordinates** for locker positions.

But Java arrays are 0-based indexed.

So we convert as follows:

Input coordinate	Array index
1	0
2	1
...	...
n	n-1

So for a locker at

```
X = 3, Y = 2
```

We convert internally to:

```
grid[1][2]
```

(row = Y – 1, column = X – 1)

Strategy (Brute Force – Correct and Efficient Enough)

Constraints are small:

- City size max = 9 × 9 = 81 cells
- Lockers max = 9

So the simplest and clearest approach:

For every grid cell:

1. Compute distance to **every locker**
2. Keep the **minimum distance**
3. Store it

Algorithm Steps

Step 1 – Create result grid

```
int[][] grid = new int[cityWidth][cityLength];
```

(rows first, columns second)

Step 2 – Loop through every city block

For each (row, col):

Convert to coordinate

x = col + 1

y = row + 1

Step 3 – Compute minimum Manhattan distance

For every locker:

```
distance = abs(x - lockerX) + abs(y - lockerY)
```

Track smallest value.

Step 4 – Store result

```
grid[row][col] = minDistance;
```

Key variables in Listing 1.7:

- **row, col:** the 0-based array indexes of the current block. grid[row][col] stores its answer.
- **currentX, currentY:** the same block in the problem's 1-based coordinates (col + 1 and row + 1).
- **minDistance:** the shortest distance found so far. It starts at Integer.MAX_VALUE, so the first locker always replaces it.

Correctness Explanation

This works because:

- Manhattan distance exactly matches problem movement rules.

- Checking all lockers guarantees nearest one is found.

- Independent per-cell calculation avoids propagation errors.

Listing 1.7 – DHL Locker Distance Grid

```
public class DHLLocker {
  static int[][] getLockerDistanceGrid(int cityLength, int cityWidth,
                                       int[] lockerXCoordinates,
                                       int[] lockerYCoordinates) {

    // Create result grid
    int[][] grid = new int[cityWidth][cityLength];

    // Loop through every cell in the city grid
    for (int row = 0; row < cityWidth; row++) {
      for (int col = 0; col < cityLength; col++) {

        // Convert array index to city coordinate (1-based)
        int currentX = col + 1;
        int currentY = row + 1;
        int minDistance = Integer.MAX_VALUE;

        // Check distance to every locker
        for (int i = 0; i < lockerXCoordinates.length; i++) {

          int lockerX = lockerXCoordinates[i];
          int lockerY = lockerYCoordinates[i];

          int distance = Math.abs(currentX - lockerX)
                       + Math.abs(currentY - lockerY);

          if (distance < minDistance) {
            minDistance = distance;
          }
        }

        // Store the minimum distance found
        grid[row][col] = minDistance;
      }
    }
    return grid;
  }

  static void printGrid(int[][] grid) {
    for (int row = 0; row < grid.length; row++) {
      for (int col = 0; col < grid[0].length; col++) {
          System.out.print(grid[row][col]);
      }
      System.out.println();
    }
  }
}
```

Tests

```
public class DHLLockerTest {
  public static void main(String[] args) {
    int cityLength = 3;
    int cityWidth = 5;
    int[] lockerX = {1};
    int[] lockerY = {1};

    int[][] grid = DHLLocker.getLockerDistanceGrid(cityLength,
                   cityWidth, lockerX, lockerY);

    DHLLocker.printGrid(grid);

    cityLength = 5;
    cityWidth = 7;
    int[] lockerX2 = {2, 4};
    int[] lockerY2 = {3, 7};

    grid = DHLLocker.getLockerDistanceGrid(cityLength,
           cityWidth, lockerX2, lockerY2);

    DHLLocker.printGrid(grid);
  }
}
```

Example Walkthrough (Example #1)

Input

```
City Length = 3
City Width = 5
```

Locker at (1,1)

Cell (3,4)

Coordinates:

```
X = 3
Y = 4
```

Distance:

```
|3 - 1| + |4 - 1|
= 2 + 3
= 5
```

So grid value becomes:

```
grid[3][2] = 5
```

This matches example output:

```
345
```

Supports multiple lockers automatically

Because we loop over all lockers and pick the minimum.

Walkthrough with two lockers (Example #2): the lockers are at (2, 3) and (4, 7). For the top-left block, X = 1 and Y = 1:

```
distance to locker (2, 3) = |1 - 2| + |1 - 3| = 1 + 2 = 3
distance to locker (4, 7) = |1 - 4| + |1 - 7| = 3 + 6 = 9
minDistance = min(3, 9) = 3  ->  grid[0][0] = 3
```

This matches the first digit of the expected output, 32345. Each block simply keeps the distance to its closest locker.

Complexity Analysis

Time complexity: O(W × L × K) for W rows, L columns, and K lockers. With the given limits, that is at most 9 × 9 × 9 = 729 operations.

Space complexity: O(W × L) for the result grid.

Follow-Up Questions

- **What if the city had 1,000 × 1,000 blocks and thousands of lockers?** Use a multi-source Breadth-First Search (see Chapter 9): put all the lockers in the queue with distance 0 and expand to neighboring blocks level by level. Each block is visited once, so the time becomes O(W × L), no matter how many lockers there are.
- **What if some blocks could not be crossed, such as buildings or rivers?** The Manhattan formula no longer applies, but the same BFS still works, because it only walks through open blocks.

Interview Tip: always read the constraints first. With at most 81 cells, a simple brute-force solution is the right answer, and saying so shows good engineering judgment.

1.8 Largest Sibling of a Number

Two non-negative integers are called siblings if they can be obtained from each other by rearranging the digits of their decimal representations. For example, 123 and 213 are siblings. 535 and 355 are also siblings.

A set consisting of a non-negative integer N and all of its siblings is called the family of N. For example, the family of 553 comprises three numbers: 355, 535 and 553.

Write a function:

```
public class Sibling {
  static int largestNumber(int N);
}
```

that, given a non-negative integer N, returns the largest number in the family of N.

For example, given N = 213 the function should return 321. Given N = 553 the function should return 553.

Assume that:

- N is an integer within the range [0 ... 10000].

In your solution, focus on correctness. The performance of your solution will not be the focus of the assessment.

Clarifying Questions

- **Can N be negative?** No. N is a non-negative integer.
- **What is the range of N?** From 0 to 10,000, so the result always fits in an int.
- **Is performance important?** The problem says to focus on correctness, so a clear solution is better than a clever one.

Brute-Force Solution

Generate every permutation of the digits (see question 4.3), convert each one to a number, and keep the largest. A number with d digits has d! permutations, so this takes O(d! × d) time. With d ≤ 5, that is at most 120 permutations, which is fast enough, but the code is longer and harder to get right, and the approach does not scale. There is a much simpler idea.

Efficient Solution

Approach

Two numbers are siblings if they contain the same digits in a different order.

The family of a number N consists of all permutations of its digits.

To find the **largest number in the family of N**, we can follow a simple strategy:

1. Convert the integer N into its individual digits.
2. Sort the digits in descending order.
3. Reconstruct the number using this order.

This works because placing the largest digits in the most significant positions always produces the maximum possible permutation.

Key variables in Listing 1.8:

- **digits:** the digits of N as characters, for example [‘2’, ‘1’, ‘3’] for 213.
- **sb:** builds the result by reading the sorted digits from last to first, that is, in descending order.

Why can we sort characters instead of numbers? The characters ‘0’ to ‘9’ have consecutive codes (48 to 57), so sorting them as characters puts them in the same order as sorting the digits as numbers.

Walkthrough: for N = 213:

```
String.valueOf(213).toCharArray() -> ['2', '1', '3']
Arrays.sort(digits)               -> ['1', '2', '3']
read from last to first           -> "321" -> 321
```

Listing 1.8 – Largest Number in the Family of N

```
import java.util.Arrays;
public class Sibling {
  static int largestNumber(int N) {
    // Convert the number to a character array
    char[] digits = String.valueOf(N).toCharArray();

    // Sort digits in ascending order
    Arrays.sort(digits);

    // Build the result in descending order
    StringBuilder sb = new StringBuilder();
    for (int i = digits.length - 1; i >= 0; i--) {
      sb.append(digits[i]);
    }

    // Convert back to integer
    return Integer.parseInt(sb.toString());
  }
}
```

Correctness Explanation

- Every sibling of N is a permutation of its digits.
- Sorting digits in descending order produces the maximum possible permutation.
- Since N is non-negative and has at most 5 digits (N ≤ 10000), integer parsing is safe.
- The algorithm always returns a valid member of N's family and guarantees it is the largest.

Tests

```
@Test
public void testSimplePermutation() {
  assertEquals(321, Sibling.largestNumber(213));
}

@Test
public void testAlreadyLargest() {
  assertEquals(553, Sibling.largestNumber(553));
}

@Test
public void testAscendingDigits() {
  assertEquals(4321, Sibling.largestNumber(1234));
}

@Test
public void testUpperBound() {
  assertEquals(10000, Sibling.largestNumber(10000));
}
```

Complexity Analysis

Time complexity: O(d log d), where d is the number of digits in N, because sorting dominates. Since d ≤ 5, this is effectively constant time for the given constraints.

Space complexity: O(d) for the character array and the StringBuilder.

Follow-Up Questions

- **Can you avoid sorting?** Yes. Count how many times each digit from 0 to 9 appears, then build the number from 9 down to 0. This is a counting sort, and it runs in O(d).
- **What if N could be close to Long.MAX_VALUE?** The largest sibling may not fit in a long. Return the result as a String, or check for overflow before parsing.
- **How would you find the smallest sibling without leading zeros?** Sort the digits in ascending order, then swap the first non-zero digit into the first position. For 3020, the answer is 2003.

> **Interview Tip:** when a problem says "focus on correctness", choose the simplest solution you can explain in one sentence: sort the digits in descending order.

YOU JUST FINISHED CHAPTER 1

40 more interview questions are waiting for you

You have seen how the book works: clarify the problem, start with brute force, reach the efficient solution, prove it with tests, and defend the Big O. The full book applies the same method to nine more chapters, including the questions interviewers love most:

- **Linked Lists:** reverse a linked list and design an LRU cache, a favorite at large tech companies.
- **Math and Logic Puzzles:** ten problems, with O(1) formulas and O(√n) prime testing.
- **Recursion:** permutations without duplicates, an O(n) Fibonacci, and the greatest common divisor.
- **Sorting and Searching:** quick sort, binary search without the overflow bug, and a meet-in-the-middle shopping problem.
- **Stacks, Queues and Hash Tables:** delimiter matching, a queue built from two stacks, and your own hash table.
- **Trees and Graphs:** binary search trees, DFS, BFS, and course recommendations in a social network.
- **Coding Challenges:** online purchases with dynamic programming, Tic Tac Toe, a filling-station simulation, and a bakery order breakdown.

Every question comes with clarifying questions, every variable explained, a step-by-step walkthrough, JUnit tests, a complexity analysis, follow-up questions (117 in total) and an interview tip. Add a Big O refresher, an appendix on clean code, generics and functional Java, and study plans for one weekend, two weeks or four weeks.

The better prepared you are, the stronger your position, not only to get the offer but also to negotiate your salary.

→ Get the full book on Amazon

Kindle and paperback · Second Edition · ISBN 9798177831381



Moises Gamio is a software engineer and senior Java developer who has sat on the candidate side of the table at startups and large companies. He wrote down every question, solved it again, and tested it, so you do not have to start from zero.

Found this sample useful? Share it with a colleague who has an interview coming up.

As an Amazon Associate I earn from qualifying purchases.